

Oppgave 1.

Informasjonssikkerhet omhandler hvordan vi sørger for at informasjonen vår og andre sin, er sikker. Det ligger litt i navnet. «Informasjon» i seg selv sier ikke så mye om hva dette egentlig er, og vi kan se på det som at informasjon har mange nivåer av hvor viktig den er å holde sikker. Hva du spiste til middag i går er kanskje ikke så viktig for deg å holde hemmelig, men om du sitter på informasjon om hvor atomvåpen oppholder seg, så er det kanskje greit at det ikke er temaet rundt frokostbordet. For å kunne forklare bedre hvordan informasjon bør sikres kan vi se nærmere på CIA-modellen. CIA står for Confidentiality (konfidensialitet), Integrity (integritet) og Availability (tilgjengelighet).

For å unngå at informasjonen vår lekkes til uvedkommende, er det viktig å beskytte dataen hvor dette er lagret og eventuelt når det skal sendes. Dette kan f.eks. gjøres ved å kryptere meldinger med en krypteringsnøkkel, for dette gjør det vanskelig/umulig, avhengig av hvilken kryptering du bruker, for uvedkommende å kunne lese klarteksten - altså innholdet i meldingen. Når klarteksten er kryptert kaller vi innholdet for chifferskrift og dette skal være umulig å lese med mindre man har den tilhørende dekrypteringsnøkkelen.

Å begrense den fysiske tilgangen er minst like viktig og kan f.eks. gjøres via passord, fingeravtrykkleser eller en HW-nøkkel (hardware-key). Alle ansatte i en bedrift bør ikke ha samme mulighet til å komme seg inn hvor som helst - da er hvert fall ikke det en bedrift jeg vil være kunde hos eller ansatt i. Det er også smart å kunne loggføre hvilke ansatte som låser seg inn hvor, slik at man har en fullstendig oversikt over dette ved behov.

At informasjonen vår blir bevart med integritet går ut på hvor mye vi kan stole på at informasjonen vi mottar ikke kan ha blitt endret underveis, som f.eks. ved det vi kaller for et "man-in-the-middle attack". Så lenge man benytter en bra krypteringsalgoritme, som f.eks. RSA, så bør ikke dette være et problem man støter på - hvis det er gjort riktig. Du kan også bruke sjekksummer som er en kort kodesnutt som genereres og sendes sammen med dataene, og kan da sammenlignes med en ny sjekksum som genereres på nytt hos de som mottar dataene. Dersom sjekksummene ikke er like, tyder dette på at endringer er gjort underveis og at integriteten av dataene er borte.

Sist, men ikke minst, er det veldig viktig at dataene kun er tilgjengelig for de som er autorisert til dette, og at de har mulighet til å raskt kunne endre informasjonen ved behov. Det er ekstremt viktig at uvedkommende ikke har tilgang til å innhente informasjon om dataene.

Oppgave 2.

Nasjonale myndigheter har, dessverre, mange metoder og verktøy de kan bruke for å samle inn diverse digitale spor om en privatperson. Geolokasjon er blitt en del av ekstremt mange sosiale medier, eller f.eks. muligheten for å "sjekke inn" på hvilket hotell du bor på i ferien din og deretter publisere det på din offentlige profil. Dette er data du har synliggjort for alle, da også myndighetene, som kan bruke dette for å spore bevegelsene dine. Dette er noe mange ikke tenker over fordi de ikke ser problemet med det når de ikke har gjort noe galt.

Alle meldinger og kommentarer du legger igjen på innlegg eller forum er, uten at du er klar over det, en enkel mulighet til å kartlegge nettverket ditt og dra konklusjoner i hvem du omgås med. Tenk deg om før du kommenterer alt og alle, selv med

ytringsfriheten på din side.

Det skumleste er nok alle bildene og videoene som blir delt, og har blitt i mange år allerede. Uten å gå inn på hvor skummelt det er at andre privatpersoner har tilgang til bilder av deg og familien din, så er dette også en gylden mulighet for myndighetene å knytte deg til spesifikke hendelser eller steder ved bruk av ansiktsgjenkjenning.

Plutselig blir det vanskelig for deg å forsvare hvorfor du var utenfor Narvesen kl.

03:30 natt til søndag, rett før en slåsskamp brøt ut.

Politiregisterloven §65 omhandler hvordan politiet behandler personopplysninger ifbm. jobben de gjør. Denne paragrafen tar for seg hvilken adgang politiet har fått til å samle inn og bruke dine personopplysninger til å f.eks. forebygge, etterforske eller avdekke straffbare handlinger. Dette gjelder altså ikke informasjon som ligger bak passordbeskyttelse, men så å si alt annet.

Dette vil nok føre til at overvåkingen av privatpersoner vil øke betraktelig, om den ikke allerede har gjort det. Privatlivet vil på mange måter bli innskrenket ettersom større mengder av personlige data nå kan samles inn og analyseres av myndighetene.

Nedkjølingseffekten vil her referere til at privatpersoner kan endre sin adferd av frykt for å bli overvåket eller få konsekvenser. Dette skaper flere negative konsekvenser, som f.eks. at personer føler de må sensurere seg selv og aktivt unngår å uttrykke sine meninger offentlig eller i diskusjoner på sosiale medier. Her kan man jo tenke på om dette indirekte går utover ytringsfriheten vår, og om §65 ble vedtatt for blant annet nettopp dette.

Når vi føler oss overvåket så blir man gjerne mer forsiktig med hva man sier høyt eller skriver på nett. Det å skulle kritisere myndighetene f.eks. i en politisk setting vil nok sitte langt inne hos noen som er redde for å bli misforstått. At folk heller brenner inne med meningene sine vil svekke demokratiet, noe som ikke er bra.

Lover og regler som §65 har naturligvis mange etiske problemstillinger ved seg, og her tenker jeg det først og fremst går hånd i hånd med spørsmål rundt personvern og samtykke. Det er viktig at vi har retningslinjer for hva slags data som kan samles inn, hvordan den samles inn og hvordan den brukes. Privatpersonene som får sin data samlet inn bør bli varslet om at dataene deres blir brukt og ha mulighet til å gi sitt samtykke, med mindre man er direkte knyttet til en straffesak.

Dette kan selvfølgelig bidra til økt sikkerhet for veldig mange i flere situasjoner, og trolig hjelper slike ordninger på effektiviteten i arbeidet hos politiet, men jeg velger å holde meg litt kritisk til sånt.

Oppgave 3.

Stuxnet var en dataorm med kinetisk utfall, altså at det kunne manipulere og sabotere fysiske prosesser og hadde potensialet til å forårsake enda større fysisk skade enn det gjorde.

Ifølge en avisartikkel fra samme tid som viruset brøt ut har Lawrence Langer, som var en verdensledende ekspert på datasikkerhet, omtalt viruset som "imponerende" og sagt at det var "skrevet på en måte man aldri før har sett" (Dagbladet, 2010).

Maskinene som var smittet av Stuxnet hadde ikke internettilgang så konklusjonen var at infeksjonen kom fra smittede USB-sticks, og at det ikke kunne smitte videre.

Stuxnet spredde seg likevel gjennom 4 ukjente "zero day" exploits av LNK fil-formatet og infiserte seg dermed videre i LAN'et fra SCADA maskinene til Siemens sine PLC

kontrollere via en feil i WinCC databasen (og dårlig sikkerhet fra Siemens sin side) (Østby, 2024).

En av årsakene til at dette var et skremmende virus kommer av at de smittede SCADA maskinene styrte atomreaktorer. Viruset inneholdt også kode som manipulerte roteringshastigheten på sentrifugene som ble brukt til å skille uran, i atomanlegget til Iran. I avisartikkelen jeg tidligere nevnte har Lawrence påpekt at “det vil ta Iran to år å komme tilbake der de var før viruset slo til” og at “dette var nesten like effektivt som å bombe atominstallasjonene” (Dagbladet, 2010). Det sier kanskje litt om skaden som ble gjort, men også hva som kunne ha skjedd om det ikke hadde blitt oppdaget.

Det var en del spekulasjoner om hvem som stod bak Stuxnet og i 2012 avslørte flere aviser at det var NSA, CIA og Israelsk etterretning som stod bak viruset. Målet med Stuxnet var visstnok å infisere atomanleggene i Iran, og skape forurenset uran slik at iranske myndigheter sluttet å forske på det (Østby, 2024).

Mange stusset på hvor enkelt informasjon som omhandlet hemmelige militære operasjoner, hadde blitt “lekket” til offentligheten, og flere var nok skeptiske til om dette var bevisst eller ikke.

For å se litt nærmere på hvilke lignende trusler vi kan støte på i vår tid, Stuxnet skjedde tross alt for 14 år siden, må vi se på alt som er nytt i 2024 kontra 2010. Verden har jo utviklet seg en god del på disse årene og spesielt innenfor teknologiens verden. Jo mer utbredt teknologien blir, jo flere og større trusler følger med - i tillegg til de “vanlige” konsekvensene av angrep/krig.

En stor innvirkning her er nok AI og hvilken rolle den kan spille i et potensielt datavirus, eller utviklingen av det. Stuxnet ble jo ikke oppdaget før ett år etter første maskin ble infisert pga. et rootkit som klarte å holde det skjult så lenge. Se for deg at samme virus ligger nå på din private maskin, og har allerede vært der i flere år fordi den som utviklet viruset har implementert AI til å skjule det enda bedre!

Et annet scenario kan f.eks. være at politiet/myndighetene/staten (en pålitelig kilde, for de aller fleste) får maskinene sine infisert med et datavirus som ikke blir oppdaget. Plutselig blir AI-genererte videoer av kongen eller statsministeren vår streamet til alle mulige skjermer over hele landet, hvor de advarer folk mot f.eks. enda en større og verre pandemi, en eventuell krig så de anbefaler alle å evakuere snarest. Om det enten er propaganda eller reelle trusler de faktisk kommer med, så er det skummelt nok hva resultatet kan være når over 5,4 millioner mennesker blir hysteriske.

Oppgave 4.

I symmetrisk kryptering bruker vi samme nøkkel til å først kryptere dataene og deretter dekryptere dem, så nøkkelen som brukes må altså være kjent for både avsender og mottaker på forhånd. Et eksempel på en symmetrisk krypteringsalgoritme er AES (Advanced Encryption Standard).

Dette er faktisk den foretrukne krypteringsmetoden å bruke i dag dersom du skal bruke en symmetrisk algoritme, blant annet fordi den ikke kan knekkes med dagens teknologi (Østby, 2024).

AES blir kalt for en blokkchiffer fordi den har en fast blokkstørrelse på 128 bits, men

med støtte for nøkkellengder på 128, 192 og 256 bits. Dette vil si at dataene som skal krypteres blir delt opp i blokker på 128 bits, og behandles hver for seg. Dersom dataene ikke er eksakt en multiplum av 128 bits må den siste blokken paddes slik at den fylles til 128 bits, ved å f.eks. legge inn en sekvens med identiske bytes som hver tilsvarer lengden på paddingen. Hvor mange runder med kryptering hver nøkkel må gjennom, kommer an på hvor mange bits den består av.

Asymmetrisk kryptering er naturligvis ulik den symmetriske varianten, og hovedforskjellen er nøklene som brukes. Denne krypteringsmetoden består av en public key og en private key, som brukes til å kryptere/dekryptere klartekst/chiffertekst. Dette kalles asymmetrisk fordi den ene nøkkelen ikke kan brukes til det samme som den andre.

Et eksempel på en slik algoritme er RSA (Rivest, Shamir, Adleman - utviklerne av denne algoritmen), som er basert på faktoriseringen av to store primtall. Det er veldig viktig at private key kun er kjent for eieren da det er denne som blir brukt til å dekryptere og signere meldinger, mens public key kan være offentlig kjent. Dette er en algoritme som ofte blir brukt til digitale signaturer, og er en super måte å sørge for at dataene man sender beholder sin integritet under sending.

Algoritmen bruker eksponenter for å kryptere klarteksten i blokker, hvor hver blokk har en binær verdi som er mindre enn et tall (n). Både avsender og mottaker må kjenne til denne verdien, samt at avsenderen må kjenne til verdien av e , og mottaker verdien av d . Sistnevnte er det vi kaller for RSA-signering.

Formelen for å kryptere en melding med RSA ser slik ut: $C = M^e \bmod n$

Formelen for å dekryptere en melding med RSA ser slik ut: $M = C^d \bmod n$

Symmetrisk kryptering krever altså sikker nøkkeldeling, men bruker samme nøkkel til både kryptering og dekryptering, samt at den er en rask og effektiv krypteringsmetode. Asymmetrisk kryptering krever ingen sikker nøkkeldeling, og sørger for integritet ved signering, men har til gjengjeld en mer langsom prosess enn symmetrisk kryptering.

Dersom vi kombinerer begge disse typene kan vi likevel oppnå både sikker og effektiv beskyttelse av dataene på samme tid. Asymmetrisk kryptering blir ofte brukt til å utveksle symmetriske nøkler sikkert, slik at man deretter kan bruke symmetrisk kryptering for å faktisk sende over dataene på en raskere og mer effektiv måte.

Oppgave 5.

Internet of Things (IoT) er det vi kaller nettverket av alle fysiske apparater som er tilknyttet internett, og som har mulighet til å samle, motta og dele data. Dette kan f.eks. være smartklokker, overvåkningskameraer, trafikklys, parkeringssensorer og egentlig alle apparater som er utstyrt med sensorer og programvare for å kommunisere med hverandre, og systemer over internett. Mange av disse "smart apparatene" har likevel lite til ingen sikkerhet, til tross for at flere og flere husstander kjøper dem, noe som naturligvis skaper en del sikkerhetsutfordringer.

Et eksempel kan være at du kjøper et overvåkningskamera som har et standardpassord fra fabrikken, aka vil alle slike kameraer denne fabrikken lager ha samme passord. Dersom du ikke endrer dette passordet kan det resultere i en helt ny Mirai Botnet situasjon (Zieniūtė, 2024).

Hackere kan nemlig finne ut av hva dette passordet er og hardkode det inn i et virus,

som da gir de tilgang til kameraet ditt og alt det filmer. Dette gir overvåkningskamera en helt ny betydning, som mange ikke er klar over engang.

Mange av disse apparatene samler også inn store mengder av data om deg, vanene dine, når du kommer hjem fra jobb og når du drar på ferie. Dette er mye informasjon som kan brukes mot deg dersom uvedkommende får tak i det. Mindre eksempler kan være at "smart låsen" på døren din blir misbrukt av en hacker, og du blir låst inne som et gissel i et ransomware angrep, men istedenfor å true med å slette filene dine så truer de heller med å aldri slippe deg ut. Da er det bra dumt å ikke ha en måte å overstyre apparatene på.

Jeg har ikke lyktes med å finne noen sikkerhetsprotokoller eller retningslinjer for hvordan dataen sikres på tvers av slike apparater, så jeg spekulerer med at dette skyldes at det ikke er noen sikkerhet rundt dette i det hele tatt.. Per dags dato. Her snakker vi gyldne muligheter for man-in-the-middle angrep, og lammelse av både apparater og nettverk via DDos-angrep også.

Tiltak for å sikre deg mot noe av det jeg har nevnt:

- Endre passord på alle smart apparater du kjøper, og er det ikke mulig så ikke kjøp det.
- Ikke gjør dagligdagse oppgaver avhengig av apparatene, for da er løpet kjørt om du mister tilgangen til de eller uvedkommende misbruker tilgangen.
- Ikke ha automasjoner satt til faste tidspunkt for når du drar på jobb og hvor lenge du blir borte - det er som å be om å komme hjem til ransaket hus. Du lærer opp apparatene til å gjenkjenne vanene dine, og det vil du ikke at uvedkommende skal få tak i.
- Selv om smart apparater kan være spennende å bruke, hold deg for smart for å bruke dem.

Oppgave 6.

Penetrasjonstesting er en metode vi kan bruke for å evaluere om en bedrift har nødvendige sikkerhetstiltak på plass, og eventuelt identifisere sårbarheter og sikkerhetshull som kan utnyttes av potensielle angripere. Dette foregår ved å simulere et ondsinnet angrep mot et nettverk, IT-system eller en applikasjon, hvor vi får muligheten til å se hvor godt disse står imot slike angrep. Det kan være sårbarheter som f.eks. utdatert programvare, mangler i tilgangskontroll eller direkte svakheter i koden.

Ved å bruke ulike verktøy kan vi f.eks. identifisere åpne porter som ikke burde være åpne, teste sårbarheten for infeksjonsangrep eller utføre brute force-angrep på forskjellige passord. Å identifisere eventuelle sårbarheter gir bedriftene verdifull innsikt i hvilke risikoer de står overfor, og hvilke tiltak som bør settes inn basert på alvorlighetsgraden, for å forbedre sikkerheten deres.

LPT (Licensed Penetration Tester) metodologien er satt sammen av fem faser for å sikre en grundig og systematisk penetrasjonstesting.

Fase 1 - Reconnaissance:

Her samles det inn så mye informasjon som mulig om alt som skal testes, f.eks. hvordan nettverket er satt opp, applikasjoner som brukes, brukerkontoer til de

ansatte og annen relevant informasjon. Dette kan gjøres passivt ved å innhente informasjon som allerede er offentlig eller aktivt ved å finne informasjonen i de forskjellige systemene. Vanligvis innhenter man informasjon både passivt og aktivt for å få et mer helhetlig bilde av sårbarhetene.

Fase 2 - Scanning:

Etter at all relevant informasjon fra første fase er samlet er det tid for å utføre scans slik at vi kan identifisere åpne porter og få en oversikt over nettverkstrafikken. Dette kan f.eks. gjøres med Nmap.

Fase 3 - Vulnerability Assessment:

Dataen som er samlet inn fra de to første fasene skal nå brukes til å identifisere potensielle sårbarheter og hvordan disse kan utnyttes av angripere. Her kan det være nyttig å bruke en kilde som National Vulnerability Database for å identifisere kjente sårbarheter og hvilken alvorlighetsgrad de har.

Fase 3 - Exploitation:

Når sårbarhetene er identifisert så er neste steg å utnytte dem ved å bruke f.eks. et verktøy som Metasploit, til å simulere et forskjellige exploits og se hvilken tilgang vi kan få ut av det.

Fase 4 - Reporting:

Den siste fasen går ut på å sette sammen en rapport som dokumenterer hva som har blitt testet og eventuelle sårbarheter som ble funnet, samt hvilke tiltak bedriften bør sette inn for å øke sikkerheten sin.

(EC-Council, 2022).

Oppgave 7.

Når data skal sendes av en person via et kommunikasjonsnett, blir dette initiert via en handling i applikasjonslaget. Før dataene kan transporteres videre til presentasjonslaget må det få med seg viktig informasjon om hvordan det skal transporteres først. TCP (Transmission Control Protocol) er et transportprotokoll som gjør det mulig å sende data over internettet, og sørger for at riktig data kommer frem til riktig mottaker, i riktig rekkefølge.

Noen av utfordringene som finnes på applikasjonslaget er SQL-Injection, Cross-Site Scripting (XSS) og Man-in-the-Middle (MitM) angrep.

SQL Injection går ut på at en angriper kan injecte skadelig SQL-kode i et skrivefelt som ikke er sikret. Dette kan føre til at uvedkommende får tilgang til databaser og muligheten til å manipulere dataen, eller i verste fall full kontroll over flere databaser.

XSS går ut på noe av det samme som SQL Injection, men her kan angripere injecte et script med skadelig kode på en nettside som andre kan besøke. Dermed får de muligheten til å f.eks. stjele informasjonskapsler eller scriptet i brukeren nettleter.

MitM angrep har jeg vært litt innom tidligere, men det er når en angriper kan lytte til kommunikasjonen mellom to parter uten at de er klar over det, og stjele/forfalske dataen.

Oppgave 8.

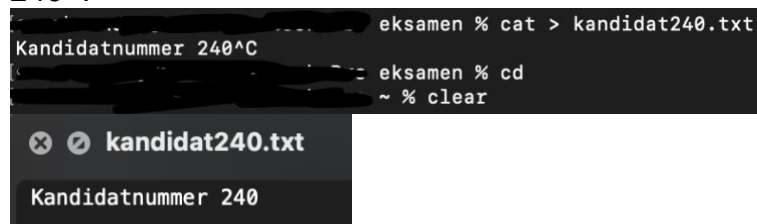
Covid resulterte i at mange bedrifter måtte sende de ansatte på hjemmekontor og som følger av dette har hjemmekontor nå blitt en mer utbredt mulighet for flere bedrifter, enn det var før pandemien. Dette kommer selvfølgelig med noen sikkerhetsutfordringer når de ansatte skal ha de samme tilgangene hjemme som de har på kontoret, i tillegg til at veldig mange benytter privat utstyr. På kontoret kunne det være kablet internett som måtte gå over VPN og en del av denne oversikten man hadde der, mistet man da alle brått måtte bli sendt på hjemmekontor uten at det ble gitt noen opplæring i hvordan de skulle jobbe hjemmefra på en sikker måte.

Den største sikkerhetstrusselen her må nok være menneskelige feil, for det er vanskelig å skulle stole 100% på at alle gjør nøyaktig som de får beskjed om, og likevel er det rom for å gjøre feil. Bare fordi man ikke vet bedre. Mye av kommunikasjonen foregår nå digitalt som gjør det enklere å falle for phishing-angrep, spesielt for de som ikke er klar over at dette i det hele tatt er en trussel. Det er likevel ikke bare den digitale sikkerheten som er truet i denne situasjonen, men for de som fremdeles er avhengig av viktige papirer, så skaper det en risiko når dette skal dras med hjem. Her er det kanskje barn som kan finne på å tegne på papirene, hunden kan spise dem, eller at naboene som stikker innom for å låne litt sukker blir nysgjerrig på hva som skjuler seg i mappen din merket "TOP SECRET".

Jeg tror at hjemmekontor er kommet for å bli og mener derfor at sikkerheten rundt dette er noe alle bedrifter må ta en aktiv stilling til, hvis de vil beholde integriteten sin ut mot kundene sine.

Oppgave 9.

Oppretter først en fil som heter «kandidat240.txt» med klarteksten «Kandidatnummer 240».



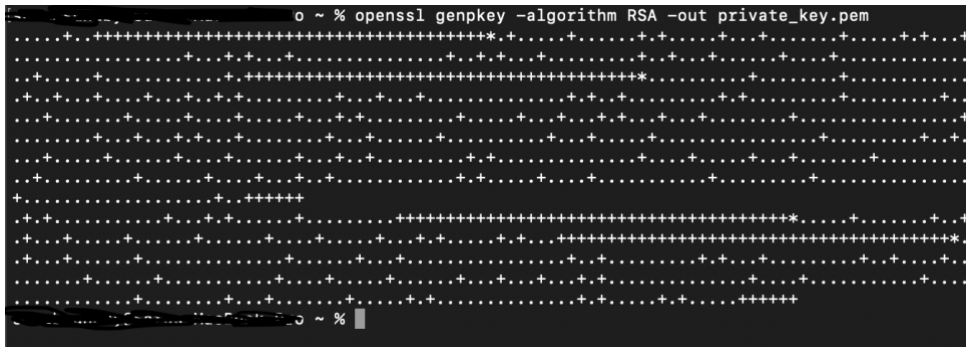
```
eksamen % cat > kandidat240.txt
Kandidatnummer 240^C
eksamen % cd
~ % clear
```

The screenshot shows a terminal window with the following content: a prompt 'eksamen %' followed by 'cat > kandidat240.txt', then the text 'Kandidatnummer 240' followed by '^C', then another prompt 'eksamen %' followed by 'cd', and finally '~ % clear'. Below the terminal window is a file icon representing 'kandidat240.txt' with the text 'Kandidatnummer 240' inside it.

Så kjører jeg følgende openssl kommando:

openssl genpkey -algorithm RSA -out private_key.pem

og får dette resultatet:



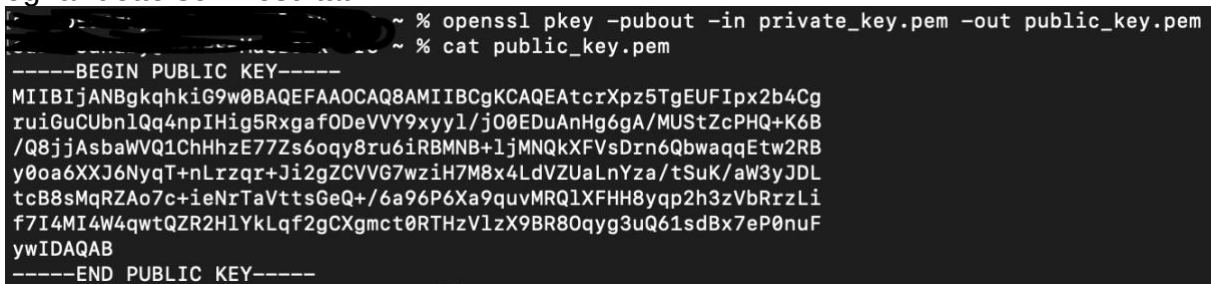
Så må jeg opprette en tilhørende public key ved å bruke denne commanden:

```
openssl pkey -pubout -in private_key.pem -out public_key.pem
```

Generering av public key skriver ikke ut noe i terminalen, så jeg bruker følgende command for å sjekke at den ble opprettet:

```
cat public_key.pem
```

og får dette som resultat:

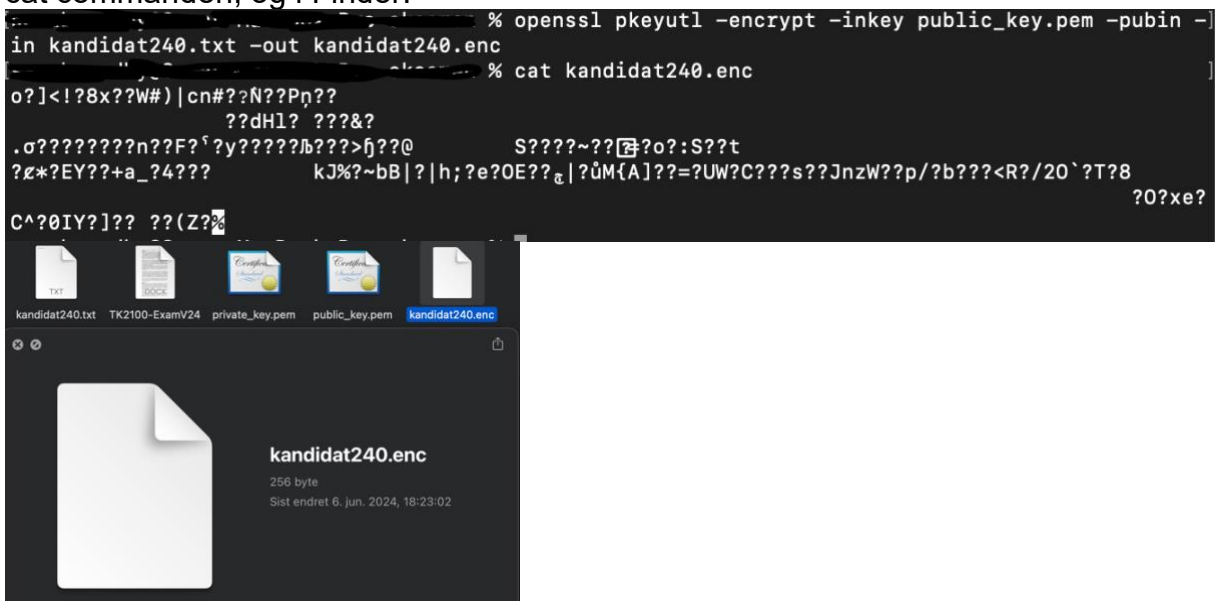


Så går jeg inn i mappen hvor begge nøkler og kandidatfilen min ligger, og kjører følgende command for å kryptere filen:

```
openssl pkeyutl -encrypt -inkey public_key.pem -pubin -in kandidat240.txt -out
```

kandidat240.enc

Dette printer heller ingenting ut til terminalen, så jeg sjekker den krypterte filen med `cat` commanden, og i Finder:



For å eventuelt dekryptere filen tilbake til klartekst bruker jeg følgende command:
`openssl pkeyutl -decrypt -inkey private_key.pem -in kandidat240.enc -out kandidat240_decrypted.txt`

og får dette som resultat:

```
root@kali:~/# cat kandidat240_decrypted.txt  
Kandidatnummer 240%
```

Oppgave 10b.

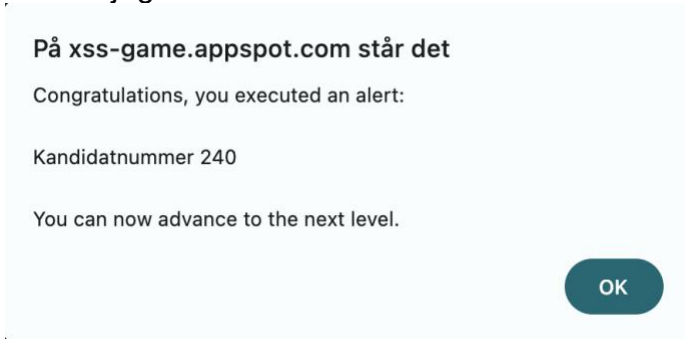
Skriver inn JavaScript kode der det står «Enter query here...»



Dette er koden jeg brukte:

```
<script>alert("Kandidatnummer 240");</script>
```

Da fikk jeg dette som resultat:



Referanseliste:

Gilbrant, J. M. (2010). *“Stuxnet” har satt Iran to år tilbake*. VG.

<https://www.dagbladet.no/nyheter/stuxnet-har-satt-iran-to-ar-tilbake/64353609>

Østby, B. (2024). *Informasjonssikkerhet: TK2101_Forelesning_04_Malware* [Forelesning, slide 13-17]. Høyskolen Kristiania i Oslo.

Østby, B. (2024). *Informasjonssikkerhet: TK2101_Forelesning_05_Kinetisk* [Forelesning, slide 39]. Høyskolen Kristiania i Oslo.

Zieniūtė, U. (2024). *What is the Mirai botnet, and how does it spread?*. NordVPN.

<https://nordvpn.com/no/blog/mirai-botnet/>

EC-Council. (2022). *Understanding the Five Phases of the Penetration Testing Process*. EC-Council.

<https://www.eccouncil.org/cybersecurity-exchange/penetration-testing/penetration-testing-phases/>